ph no : 6289449233
gmail : arijitdas.das99@gmail.com

Pandas

Deffn:

Pandas is a Python library which provides fast, flexible and expressive data structured design to make working with "relational", or "labeled data both easy and intuitive.

Things that Pandas does well:

→ Analyze data
→ Data Manipulation
→ Data cleaning
→ exploring

Creation:

Created by Wes McKinney in 2008.

Application:

→ Easy handling missing data.
→ Size mutability: columns can be inserted deleted from DataFrame and higher dimensional objects.
→ Automatic and explicit data alignment:
Objects can be explicitly aligned to a set of labels, or the user can simply ignore the labels and let Series, DataFrame etc.

What makes Pandas Unique ?

- **Key features:**

(i) Works seamlessly with structured data formats like csv & excel.

(ii) Handles missing values.

(iii) Built on Numpy ~~com~~ for fast computation.

(iv) Handles millions of rows ~~efficiently~~.

(v) Works with libraries like Matplotlib (visualization) and Scikit-learn (ML).

Real-life examples of Pandas in Action.

- **Finance:**
  - Analyzing time-series data like stock prices.
- **Retail:**
  - Tracking inventory and finding the most sold products in a store.
- **Healthcare:**
  - Analyzing patient records and outcomes from clinical trials.

Exploring Data

① **Installation of Pandas:**

```
pip install Pandas
```

② **In VS code: / Jupyter Notebook:**

```
import Pandas as pd
```

③ **Installation of open pyxl:**

Note: This is for importing the exeel file.

```
pip install openpyxl
```

④ **To load the file:**

(i) **For csv file: / Comma-Seperated Values:**

```
df = pd.read_csv("data.csv") # load the csv file.
print(df)
```

(ii) **For excel file:**

```
df = pd.read_excel("data.xlsx") # load the excel file
print(df)
```

⑤ **Kaggle (Kaggle is best for Real datasets):**

- **Website:** https://www.Kaggle.com/datasets.
- **Search for:** "csv" "missing data" "student data" "sales data".

Pandas - Analyzing DataFrames

① Viewing the Data

Printing the first 10 rows.

code ① [head( )]

```
import pandas as pd

data = pd.read_csv('data.csv')
print(data.head(10))
```

② Printing the last 10 rows.

code ② [tail( )]

```
import pandas as pd

data = pd.read_csv('data.csv')
print(data.tail(10))
```

③ Printing the information about the data / file.

code ③ [info( )]

```
import pandas as pd

data = pd.read_csv('data.csv')
print(data.info())
```

④ Printing the statistical information about the data-frame.

code ④ [describe()]

```
import pandas as pd
data = read_excel ('data.xlsx')
print(data.describe())
```

⑤ Short-cut-Visualization :

① data.head()
② data.tail()
③ data.info()
④ data.describe()

⑤ Save that file in your IDLE.

```
import pandas as pd
data = pd.read_excel('data.xlsx')
data.to_excel('file_name.xlsx')
```

Pandas - Column Transformation

① Adding columns :

(i)

```
df["colum_new"] = df["present_column"] * 0.1

df["Bonus"] = df["Salary"] * 0.1
```

↑ new column — ↑ condition

(ii)

Insert() :

```
df.insert(column_index, "column_name", data)
df.insert(0, 'Id', range(1, len(df) + 1))
```

② Accessing columns :

(i) Single column :

```
Column = df["column_name"]
Column
```

(ii) Multiple columns :

```
Columns = df[["column 1", "column 2"]]
Columns
```

③ Accessing rows :

(i) Single row by a condition :

```
row = df[df["column_name"] condition]
row
```

ex :

```
row = df[df["Salary"] > 70000]]
row
```

(ii) Accessing multiple rows by condition :

```
rows = df[(df["salary"] > 70000 & (df["salary"] < 90000)]
rows
```

(4) Updating columns :

(i)
```
df['salary'] = df['salary'] + 10000
df
```

(5) Updating rows :

```
df.loc[0, 'Salary'] = 40000
df
```

(5) Drop column :

(i) Single column :

```
df.drop('column_name', inplace = True)
```

(ii) Multiple column :

```
df.drop('column1', 'column2', inplace = True)
```

Pandas – Handling missing data

① There will always be a possibility of missing values in dataframe or in a particular column.

② That column will consists of two types of data.

(i) NaN

(ii) None

③ NaN → Not a number.

None → for objects.

Ⓐ Check if there are any missing numbers present in a data set.

→ You can check it by → `.isnull()`

→ Or you can check it by → `.isnull().sum()`

✱ isnull() → will return True or False.

- True ⇒ If missing value present.
- False ⇒ If there is no missing value.

① `df.isnull()`

② `df.isnull().sum()`

Ⓑ Process of handling null values.

→ You can totally drop that missing data/value row by ↓

`df.dropna(inplace = True)`

✱ It will drop all rows, that consists of null values.

(ii) For object / None Values,

→ you can simply write / use .ffill() or .bfill() function.

* .ffill() → fill the object in forward.
* .bfill() → fill the object in backward.

code

```
df['column_name'] = df['column_name'].ffill()
df['column_name'] = df['column_name'].bfill()
```

* Summary

1. .dropna(inplace = True)
2. .fillna(0, inplace = True)
3. .ffill() / .bfill()
4. .replace()
5. .interpolate(method="linear")

(4) You can also get a interpolate value of a specific column that consists of null value.

→ `df['column_name'].interpolate(method='linear')`

→ methods ⇒ linear, polynomial, Time.

```
df['column'] = df['column'].interpolate(method='linear')
```

→ Or you can fill with some values into the null places. by ↓

```
df.ffill(0, inplace=True)
```

* It will fill the all null places with 0.

Note

* It is not a correct approach to handle missing data.

③ **Correct approach :**

(i) → For NaN values, you can simply replace those null places with some specific values you want. fill. by ↓

```
df['column_name'] = df['column_name'].replace(np.nan, value)
```

→ You have to import numpy as np, then you have to find the mean() / average of that specific column. Then you can replace them with that particular mean value.

 # Code

```
import pandas as pd
import numpy as np
df = pd.read_excel('file.xlsx')
df['column_name'].mean() # get the mean value.
df['col_name'] = df['col_name'].replace(np.nan, value)
```

Pandas — Sorting & Aggregation

① You can sort a numerical data frame by ↓

In ascending order. (Single Column)

```
df.sort_values(by='column', aseending=True, inplae=True)
```

In diseending order. (Multiple column)

```
df.sort_values(by='column', ascending=False, inplae=True)
```

Multiple column:

```
df.sort_values(by=['column1','column2'], ascending=[False, True], inplaee=True)
```

Aggregation

- There are total 7 types of aggregate functions.
- You can get a statistical overview of your table.

→ `average = df['salary'].mean()`

→ `total = df['salary'].sum()`

→ `min_salary = df['salary'].min()`

→ `max_salary = df['salary'].max()`

→ `std_salary = df['salary'].std()`

→ `median_salary = df['salary'].median()`

→ `mode_salary = df['salary'].mode()`

Pandas — GroupBy

```
import pandas as pd

data = {
        'name': ['Arijit', 'Rahul', 'Shivam', 'Kartik', 'Varun'],
        'age': [20, 30, 20, 30, 40],
        'Salary': [40000, 30000, 60000, 70000, 10000]
}

df = pd.DataFrame(data)
print(df)
```

age	name	
20	Arijit shivam	100000
30	Rahul Kartik	100000
40	Varun	100000

multiple column

```
# single column
grouped = df.groupby('age')['Salary'].sum()
print(grouped)

# multiple column
grouped = df.groupby(['age', 'name'])['Salary'].sum()
print(grouped)
```

⇒ The groupby() method is used to split a DataFrame into groups based on the values of one or more columns.

⇒ You can perform aggregate operations like (Sum, mean, max, min, std, count).

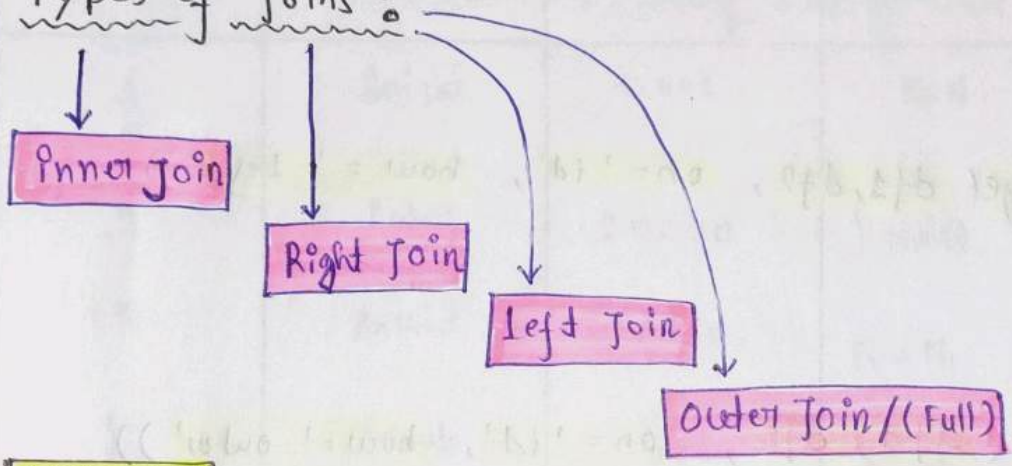
Pandas - Joins & Concatenation

Defn:

Joins are used to combine two tables / DataFrames based on a common column (like SQL Joins).

fn: merge(), concat()

Types of Joins:



1) merge()

Syntax using:

```
import pandas as pd

data1 = {
        "Id": [1, 2, 3, 4, 5],
        "name": ["Abhijit", "Rahul", "Ankit", "Aonab", "Ram"],
        "salary": [10000, 20000, 30000, 40000, 50000]
}

df1 = pd.DataFrame(data1)
print(df1)

data2 = {
        "Id": [1, 2, 3, 4, 6],
        "address": ["Dum Dum", "Durgapur", "Birati", "Barasat",
                    "Siliguri"]
```

inner join

```
print ( pd. merge ( df1, df2,  on = 'id' ,  how = ' inner' ))
```

Right Join

```
print ( pd. merge ( df1, df2,  on = 'id' ,  how = 'Right' ))
```

Left Join

```
print ( pd. merge ( df1, df2,  on = 'id' ,  how = ' left' ))
```

outer join

```
print ( pd. merge ( df1, df2,  on = 'id',  how = ' outer' ))
```

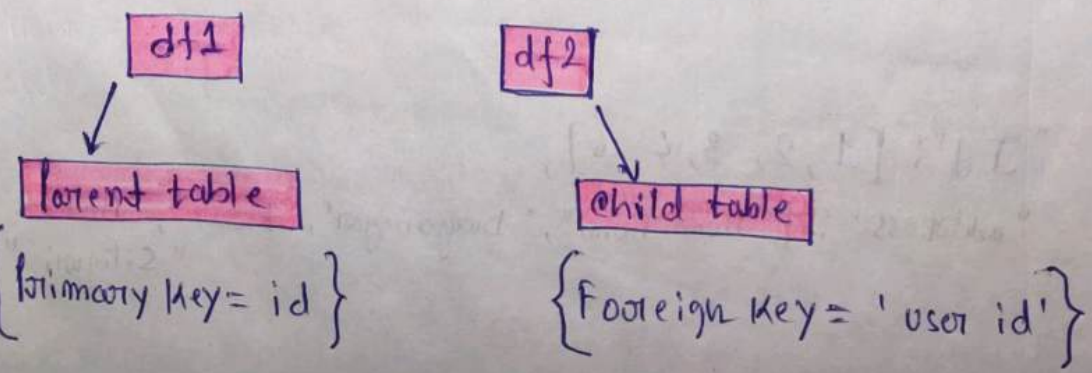
② Joining with different column names.

```
pd. merge  ( df1, df2, left_on = 'id' , Right_on = 'user_id', how = 'inner')
```

⇒ You can think of it like parent table and child table.

⇒ We are choosing Primary Key = 'id' from Parent table and Foreign Key = 'user_id' from child table.

⇒



(B) **concat()**

concat() - function used to stack the columns.

Syntax

```
print(pd.concat([df1, df2]))
```

Id	name	salary	address
1	Arijit	10000	NaN
2	Rahul	20000	NaN
3	Ankit	30000	NaN
4	Arnab	40000	NaN
5	Ram	50000	NaN
1	NaN	NaN	Dum Dum
2	NaN	NaN	Durganagar
3	NaN	NaN	Birati
4	NaN	NaN	Barasat.
6	NaN	NaN	Siliguri

Pandas - Comparing data frame.

Note: By comparing data frames, you can easily analyze data

```
import pandas as pd

data = {
        'Fruits': ['Mango', 'Apple', 'Banana', 'Orange'],
        'Quantity': [10, 15, 7, 9],
        'Price': [150, 200, 50, 90]
}

df1 = pd.DataFrame(data)
print(df1)

df2 = df1.copy()  # Same as df1 data.
print(df2)

#changing the data inside the frame df2.

df2.loc[0, 'Quantity'] = 20
df2.loc[2, 'Quantity'] = 10
df2.loc[0, 'Price'] = 200
df2.loc[2, 'Price'] = 100

print(df2)

#Now comparing two different data frame by
df1.compare(df2, keep_shape=True)
```

Table 1

	Quantity	Price
0	10	150
1	15	200
2	7	50
3	9	90

Table 2

	Quantity	Price
0	20	200
1	15	200
2	10	100
3	9	90

Pandas - Pivot Table

What is Pivot table in Pandas?

⇒ A pivot table in Pandas is used to analyze data. it allows you to →

- Rearrange data.
- Aggregate Values.
- Compare data accross multiple column.

```
import pandas as pd

table = {
        'Id' : [ 1, 2, 3, 4, 5],
        'name': ['Arijit', 'Rahul', 'Sourav', 'Ankit', 'Arnab']
        'Grades':['O', 'AA', 'A', 'B', 'e' ]
}
df= pd. DataFrame (table)
print( df)
# Pivot
print( df. pivot( index='Id' , columns='name', values='Grades'))
```

name	Ankit	Arijit	Arnab	Rahul	Sourav
Id					
1		O		AA	
2				AA	A
3	B				A
4	B				
5			e		

Applying aggregate functions:

Key Rule:

- **Index** : How you group rows.
- **Columns**: How you split columns.
- **Values**: The actual numbers on which aggregation.
- **Use**: pivot_table

```
import pandas as pd

table = {

    'Region' : ['East','East','West','West','East','West'],

    'Product' : ['A','B','A','B','A','B'],

    'sales' : [100, 150, 200, 250, 300, 400]

}

df = pd.DataFrame(table)

print( df.pivot_table( index='Region', Values='Sales',
                                    aggfune='sum'))
```

	Sales
East	550
West	850

Pandas - Melt

Deff^n Pandas .melt() function is used to transform a wide dataFrame into a long tidy format.

Code

```
import pandas as pd

df = pd.DataFrame({
        'name': ['Alice', 'Bob'],
        'Math': [85, 90],
        'Science': [88, 92]
})

melted = pd.melt(df, id_vars=['name'], value_vars=
        ['Math', 'Science'], var_name='Subject',
        value_name='Score')

print(melted)
```

Output:

variable_name → Subject

Value_name → Score

name	Subject	Score
Alice	Math	85
Bob	Math	90
Alice	Science	88
Bob	Science	92

Pandas - Handling Duplicate values.

Approach:

① Detect duplicates:

```
df.duplicated()
```

output

That will return boolean values [True / False].

- True → Duplicate row
- False → Unique row

② To count total duplicates:

```
df.duplicated().sum()
```

output

That will return the total count of duplicate values which is present in DataFrame.

③ To count total duplicates in a specific column:

```
df['column_name'].duplicated().sum()
```

output

That will return the total number of duplicates in that specific column.

Drop duplicates:

(i)
```
df_cleaned = df.drop_duplicates()
df_cleaned
```

(ii) Keep First or last

- Keeps first occurrence:

```
df.drop_duplicates(keep='first')
df
```

- Keeps last occurrence:

```
df.drop_duplicates(keep='last')
df
```

(iii) drop all:

```
df.drop_duplicates(keep=none)
df
```

(iv) Handle duplicates based on specific columns:

```
df.drop_duplicates(subset=['column_name'])
```

(*) Note
If you want to flag those duplicate values, you have to ↓

```
df['Is_duplicated'] = df.duplicated()
df
```